## 1.1.1. Calculate Momentum

13:03

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum $p$ is calculated using the formula:

$$p = m \times v$$

where:
$m$ is the mass of the object (in kilograms).
$v$ is the velocity of the object (in meters per second).

**Input Format:**

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

**Output Format:**

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases +

Explorer

calculate...

Submit

```
1  m = float(input())
2  v = float(input())
3
4  p = m*v
5
6  print(f"{p:.2f}kgm/s")
```

Debugger

Average time
**0.027 s**
26.75 ms

Maximum time
**0.053 s**
53.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 53 ms Debug

| Expected output | Actual output |
|---|---|
| 5.0 | 5.0 |
| 10.0 | 10.0 |
| 50.00kgm/s | 50.00kgm/s |

Test case 2 14 ms

Terminal Test cases

## 1.1.2. Conditional Calculation Based on the Number of Digits

00:36

Write a Python program that accepts an integer $n$ as input. Depending on the number of digits in $n$.

**Constraints:**

$1 \leq n \leq 999$

**Input Format:**

- The input consists of a single integer $n$.

**Output Format:**

- If $n$ is a single-digit number, print its square.
- If $n$ is a two-digit number, print its square root (rounded to two decimal places).
- If $n$ is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Sample Test Cases +

Explorer

condition...

Submit

```
#write your code here...
import math

# Get the input integer from the user
try:
    n = int(input())

    # Check the number of digits based on the constraints (1 <= n <= 999)
    if 1 <= n <= 9:
        # Single-digit number: print its square
        print(n ** 2)
```

Debugger

| Average time | Maximum time |
|---|---|
| 0.008 s | 0.012 s |
| 8.43 ms | 12.00 ms |

4 out of 4 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 12 ms Debug

| Expected output | Actual output |
|---|---|
| 9 | 9 |
| 81 | 81 |

Test case 2 11 ms

Test case 3 7 ms

Terminal Test cases

## 1.1.3. Days between Two Dates

00:32

Write a Python program to calculate number of days between two dates.

**Input Format:**

- The first line contains the first date in the format $YYYY - MM - DD$.
- The second line contains the second date in the format $YYYY - MM - DD$.

**Output Format:**

- An integer representing the number of days between the given dates.

**Note:**

- The first date should always be considered earlier than the second date.

Sample Test Cases +

Explorer

noOfDay...

Submit

```
1  from datetime import date
2
3  #write your code here...
4
5
6  def calculate_days():
7      try:
8          # Read the first date from input
9          date_str1 = input().strip()
10         # Read the second date from input
11         date_str2 = input().strip()
```

Debugger

Average time **0.014 s** 14.25 ms

Maximum time **0.018 s** 18.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 18 ms Debug

| Expected output | Actual output |
|---|---|
| 2014-07-02 | 2014-07-02 |
| 2014-11-02 | 2014-11-02 |
| 123 | 123 |

Test case 2 13 ms

Terminal Test cases

‹ Prev Reset Submit Next ›

## 1.1.4. Reverse a Number

00:23

Write a Python program to reverse the digits of the number and print the reversed number.

**Input Format:**

- The input is a single integer.

**Output Format:**

- The output prints a single integer, which is the reversed number.

Sample Test Cases +

Explorer

reverseN...

Submit

```
#write your code here..
# Taking integer input from the user
num = int(input())

reversed_num = 0

# Logic to reverse the digits
while num > 0:
    digit = num % 10           # Get the last digit
    reversed_num = reversed_num * 10 + digit  # Append digit to
reversed num
```

Debugger

Average time **0.008 s** 8.20 ms

Maximum time **0.010 s** 10.00 ms

2 out of 2 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 10 ms Debug

| Expected output | Actual output |
|---|---|
| 5367 | 5367 |
| 7635 | 7635 |

Test case 2 9 ms

Terminal Test cases

## 1.1.5. Multiplication Table

00:28

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

**Input Format:**

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

**Output Format:**

- Print the multiplication table for the given number in the format:

```
<number> x <multiplier> = <result>
```

**Note:**

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases +

Explorer

multiplica...

Submit

Debugger

```python
#write your code here...
# Step 1: Take an integer as input from the user
number = int(input())

# Step 2: Use a for loop to iterate from 1 to 10
for i in range(1, 11):
    # Step 3: Calculate the result
    result = number * i

    # Step 4: Print the output in the specified format: <number> x <multiplier> = <result>
```

Average time **0.009 s** 8.75 ms

Maximum time **0.011 s** 11.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 11 ms Debug

| Expected output | Actual output |
|---|---|
| 8 | 8 |
| 8 x 1 = 8 | 8 x 1 = 8 |
| 8 x 2 = 16 | 8 x 2 = 16 |
| 8 x 3 = 24 | 8 x 3 = 24 |
| 8 x 4 = 32 | 8 x 4 = 32 |
| 8 x 5 = 40 | 8 x 5 = 40 |

Terminal Test cases

## 2.1.1. List operations

08:04

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add**: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display**: Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit**: Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice".Ensure that the program continues to prompt the user until they choose to quit (option 4).

Sample Test Cases +

Explorer

listOps.py

Submit

```
# write the code..
def main():
    my_list = []

    while True:
        # Display the Menu
        print("1. Add")
        print("2. Remove")
        print("3. Display")
        print("4. Quit")

```

Debugger

| Average time | Maximum time |
|---|---|
| 0.099 s | 0.150 s |
| 98.67 ms | 150.00 ms |

2 out of 2 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 77 ms Debug

| Expected output | Actual output |
|---|---|
| 1. Add | 1. Add |
| 2. Remove | 2. Remove |
| 3. Display | 3. Display |
| 4. Quit | 4. Quit |
| Enter choice: 1 | Enter choice: 1 |
| Integer: 11 | Integer: 11 |

Terminal Test cases

‹ Prev Reset Submit Next ›

## 2.1.2. Dictionary Operations

01:54

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

**Operations to be Performed:**

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

**Input and Output Format:**

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

**Note:**

- All operations must be performed using dictionary methods.
- Perform deletion only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Explorer

DictOper...

Submit

Debugger

```
1  # Initial dictionary with 10 predefined records
2  student = {
3      1: "Amit",
4      2: "Riya",
5      3: "Kiran",
6      4: "Neha",
7      5: "Arjun",
8      6: "Pooja",
9      7: "Rahul",
10     8: "Sneha",
11     9: "Vikram",
```

| Average time | Maximum time |
|---|---|
| 0.028 s | 0.033 s |
| 28.25 ms | 33.00 ms |

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 33 ms Debug

| Expected output | Actual output |
|---|---|
| Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'} | Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'} |
| 11 | 11 |
| Suresh | Suresh |
| After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahu | After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rah |

Terminal Test cases

## 3.1.1. Numpy Array Operations

00:50

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

**Input Format:**

- The first line contains two space-separated integers, $r$ and $c$, representing the number of rows and the number of columns.
- The next $r$ lines contain $c$ space-separated integers representing the elements of the array, entered row by row.

**Output Format:**

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

**Note:** Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases +

Explorer

numpyarr...

Submit

```
1  import numpy as np
2
3  # write your code here...
4
5  def solve():
6      # Read the number of rows (r) and columns (c)
7      try:
8          r, c = map(int, input().split())
9      except ValueError:
10         return
11
```

Debugger

| Average time | Maximum time |
|---|---|
| 0.019 s | 0.032 s |
| 19.08 ms | 32.00 ms |

2 out of 2 shown test case(s) passed

10 out of 10 hidden test case(s) passed

Test case 1 32 ms Debug

| Expected output | Actual output |
|---|---|
| 3 4 | 3 4 |
| 1 2 3 4 | 1 2 3 4 |
| 5 6 7 8 | 5 6 7 8 |
| 9 10 11 12 | 9 10 11 12 |
| [[ 1 2 3 4] | [[ 1 2 3 4] |
| [ 5 6 7 8] | [ 5 6 7 8] |

Terminal Test cases

## 4.1.1. Pandas - series creation and manipulation

18:27

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

**Input Format:**

- The user should enter a list of numbers separated by space when prompted.

**Output Format:**

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases +

Explorer

seriesMa...

Submit

```
1  import pandas as pd
2
3  # Take inputs from the user to create a list of numbers
4  numbers = list(map(int, input().split()))
5
6  # Create a Pandas series from the list of numbers
7  s = pd.Series(numbers)
8  # Grouping by even and odd numbers and calculating the mean
9  grouped = s.groupby(s % 2 == 0).mean()
10
11 # Display the mean of even and odd numbers with labels
```

Debugger

Average time **0.019 s** 18.83 ms

Maximum time **0.047 s** 47.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 47 ms Debug

| Expected output | Actual output |
|---|---|
| 1 2 3 4 5 6 7 8 9 10 | 1 2 3 4 5 6 7 8 9 10 |
| Mean of even and odd numbers: | Mean of even and odd numbers: |
| Odd 5.0 | Odd 5.0 |
| Even 6.0 | Even 6.0 |
| dtype: float64 | dtype: float64 |

Terminal Test cases

## 4.1.2. Dictionary to dataframe

48:36

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

**Create the DataFrame:**

- Convert the dictionary to a Pandas DataFrame.

**Add a new row:**

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

**Modify a row:**

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

**Delete a row:**

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

**Add a new column:**

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

**Modify a column:**

- Convert names to uppercase

Explorer

datafram...

Submit

```
1  import pandas as pd
2
3  # Provided dictionary of lists
4  data = {
5      'Name': ['Alice', 'Bob', 'Charlie'],
6      'Age': [25, 30, 35],
7  }
8
9  # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
11
```

Debugger

Average time **0.160 s** 159.50 ms

Maximum time **0.195 s** 195.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 195 ms Debug

| Expected output | Actual output |
|---|---|
| Original DataFrame: | Original DataFrame: |
| Name Age | Name Age |
| 0 Alice 25 | 0 Alice 25 |
| 1 Bob 30 | 1 Bob 30 |
| 2 Charlie 35 | 2 Charlie 35 |
| New name: Susan | New name: Susan |

Terminal Test cases

‹ Prev Reset Submit Next ›

## 4.1.3. Student Information

10:43

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is **'B'**).

**Note:**
Refer to the displayed test cases for better understanding.

Sample Test Cases +

Explorer

studentin...

Submit

```
1  import pandas as pd
2
3  # Read the text file into a DataFrame
4  file = input()
5  data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age",
   "Grade"])
6
7
8  # write your code here..
9  print("First five rows:")
10 print(data.head(5))
```

Debugger

Average time **0.084 s** 83.50 ms

Maximum time **0.123 s** 123.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 123 ms Debug

| Expected output | Actual output |
|---|---|
| studentdata.txt | studentdata.txt |
| First five rows: | First five rows: |
| Name Age Grade | Name Age Grade |
| 0 John 25 A | 0 John 25 A |
| 1 Alin 22 B | 1 Alin 22 B |
| 2 Emma 24 A | 2 Emma 24 A |

Terminal Test cases

## 5.1.1. Stacked Plot

03:33

Create a stacked area plot to visualize the temperature variations for three different cities (City A, City B, and City C) across the months of the year. The temperature data is provided for each city in the editor.

Your task is to:

- Create a stacked area plot using the data.
- Label the x-axis as "Month", the y-axis as "Temperature", and provide the title "Temperature Variation" for the plot.
- Display the plot showing the temperature variation for each city throughout the months of the year.

Sample Test Cases +

Explorer

stackedpl...

Submit

```
1  import matplotlib.pyplot as plt
2  import pandas as pd
3
4  # Data for Months and Temperature for three cities
5  data = {
6      'Month': ['January', 'February', 'March', 'April', 'May', 'June',
   'July', 'August', 'September', 'October', 'November', 'December'],
7      'City_A_Temperature': [5, 7, 10, 13, 17, 20, 22, 21, 18, 12, 8, 6],
8      'City_B_Temperature': [2, 3, 5, 6, 10, 14, 16, 17, 12, 9, 5, 3],
9      'City_C_Temperature': [3, 4, 6, 8, 9, 12, 15, 14, 10, 7, 4, 2]
10 }
```

Debugger

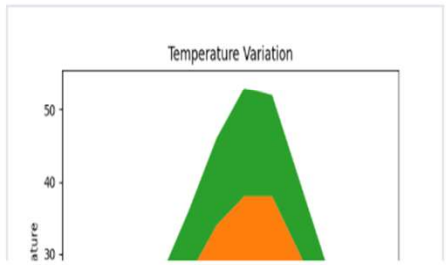
Average time 0.589 s 589.00 ms

Maximum time 0.589 s 589.00 ms

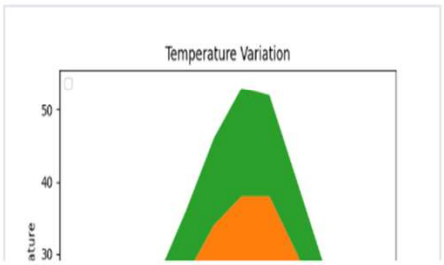
1 out of 1 shown test case(s) passed

Test case 1 589 ms Debug

Expected output




Actual output




Terminal Test cases